

DHCP starvation and rogue DHCP

Follow-along: play the attacker, then the defender

Overview

Definition: DHCP lease

The address, netmask, and options a server allocates to one client for a bounded time. The client picks none of them: it broadcasts a request onto the LAN and installs what comes back, including the address it will use as its default gateway.

Definition: DHCP starvation

Completing DHCP handshakes under invented hardware addresses until the server has allocated every address it has. Each fake client holds a real lease, so the server has nothing left to offer a machine that asks afterwards.

A host that arrives on a LAN with no address of its own broadcasts a request for one, and a DHCP server answers with a **DHCP lease**. **DHCP starvation** attacks the finite thing in that exchange: the server's range of addresses. One host on the LAN completes handshake after handshake under hardware addresses it makes up on the spot, and every completed handshake takes an address out of the range for as long as the lease runs.

An empty pool denies service on its own, but the reason to drain it is what it lets you do next. With the real server unable to answer, the attacker starts a DHCP server of its own and is the only one left that can reply, so it dictates what every host that asks from then on installs as its default gateway: the attacker's own address. The victim then sends everything bound for another subnet to the attacker first, and its connectivity looks unchanged. Part 1 builds that position by hand, one stage at a time. Part 2 closes it at the switch, the one device every DHCP message on this LAN crosses.

Running this lab in the TUI

You drive the lab from the MiniLabs launcher. Clone it if the machine does not have it, and pull if it does, because handouts and lab scripts change together:

```
git clone https://github.com/nzRichie/MiniLab.git ~/MiniLab # first time
cd ~/MiniLab && git pull                                     # already cloned
```

Start the launcher from that directory:

```
cd ~/MiniLab && ./minilabs
```

If it exits with a Docker error, follow the setup in the repository's `README.md` and run it again.

The root menu lists one lab per row. Select **DHCP starvation and rogue DHCP** to open its actions:

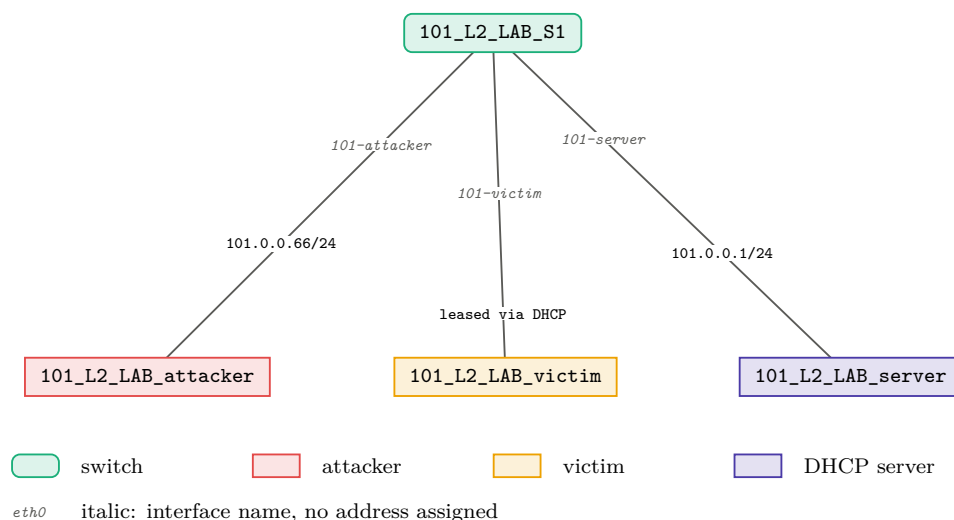
- **Spawn** creates this lab's switch, server, victim, and attacker, wires them, and leaves the victim holding a lease from the real server. Wait for the panel to turn green before you go on. The first spawn of a lab also builds its images, which takes a few minutes; later spawns take seconds.

- **Status** prints each host's addressing and the gateway the victim currently trusts, which is the one value that says whether the rogue has taken over.
- **Shell command for a node** asks which node (server, victim, attacker, S1) and prints the `docker exec -it ...` line to paste into your own terminal; each step below names the node it runs on.
- **Reset to baseline** stops the flood and the rogue server, drops your switch rules, refills the real pool, and re-leases the victim, returning the lab to its clean Part 1 state without tearing it down.
- **Teardown** removes the lab.
- **Open handout** opens this document.

Spawn the lab now, then work through Part 1 in your own terminal. **Back** (or Esc) returns to the lab list.

The setup

Three hosts and one switch on a single flat broadcast domain, 101.0.0.0/24:



Each node is labelled with the container name you pass to `docker exec`, and each end of a link with the address on that interface. The victim's end says where its address comes from instead of naming one, because it has no fixed address; that is what makes it the target. The same addressing, to copy from:

Role	IP	Container
server	101.0.0.1	101_L2_LAB_server
attacker	101.0.0.66	101_L2_LAB_attacker
victim	leased over DHCP	101_L2_LAB_victim
switch	(none)	101_L2_LAB_S1

All three hosts attach to the same Open vSwitch bridge, `br0`, inside `101_L2_LAB_S1`. Every host's NIC is called `101-S1`; the switch port each one lands on is named after the host (`101-server`, `101-victim`, `101-attacker`).

The server runs a real ISC `dhcpcd`. It leases addresses from `101.0.0.100` to `101.0.0.120` and hands out `101.0.0.1`, itself, as the default gateway. Leases run for two hours, so nothing expires

and frees an address while you work.

The attacker carries a stock kit; there is no bespoke exploit to run:

- **dhcpc-starve**: completes DHCP handshakes with invented MAC addresses. You give it the interface, and optionally how many addresses to take.
- **dhcpcd**: a second copy of the same ISC server the victim trusts. You write a short config for it and run it as the rogue server.
- **tcpdump**: read the DHCP exchange, and watch the traffic you divert.
- **udhcpc**: a DHCP client, used here to act as a machine joining the LAN.

The switch container adds **ovs-vsctl** and **ovs-ofctl**, which read and write the bridge's ports and its forwarding table. Part 2 uses both. IP forwarding is already on in every container, set when it was created; it is what lets the attacker relay a victim's traffic once it is the gateway.

Part 1 needs three terminals open at once: one on the victim and two on the attacker. Part 2 adds one on the switch and one on the server. Every command block below opens with a comment naming the container its commands go in, and each block that starts a new session shows the **docker exec** that opens it:

```
# 101_L2_LAB_victim: this is how every step below opens a shell
docker exec -it 101_L2_LAB_victim bash
```

Assessment. This lab is graded on eighteen questions, marked inline below as **Question N**, each placed at the step whose output it asks about. Answer all eighteen in a separate document as you work through the steps. Every question is answered from what you see on screen at that step; none needs anything beyond the lab in front of you.

Part 1: the attack

1. **Confirm the honest baseline.** The victim leased an address when the lab came up. Read what it is running on, and the hardware address it asked with:

```
# 101_L2_LAB_victim: the victim, opened from your own terminal
docker exec -it 101_L2_LAB_victim bash
ip -4 addr show 101-S1          # an address out of the server's range
ip route show default          # the gateway the server told it to use
cat /sys/class/net/101-S1/address # this host's hardware address
```

Write down all three. Nothing in the victim's configuration names an address or a gateway. Now read the server's side of the same lease:

```
# 101_L2_LAB_server: one command, run from your own terminal
docker exec 101_L2_LAB_server cat /var/lib/dhcp/dhcpd.leases
```

Question 1. Find the lease block for the address the victim is holding. Name the field in that block that ties the address to this victim rather than to any other machine on the LAN, and give its value. Then say what the server does with a request that carries a value it has no lease block for.

2. **Recon: read an offer.** The attacker needs two things out of a real exchange: which server it has to out-compete, and which gateway that server is telling clients to use.

Definition: DHCP option

A numbered, named field carried in a DHCP message alongside the address itself. Everything a client receives beyond its own address arrives as an option, and `tcpdump -v` prints each one by name and number.

Both are **DHCP options** in the server's reply, so watch one exchange go past. In your first attacker shell, start a capture that decodes DHCP:

```
# 101_L2_LAB_attacker: your first attacker shell, left running
docker exec -it 101_L2_LAB_attacker bash
tcpdump -n -v -i 101-S1 'udp and port 68'
```

In a second attacker shell, trigger a single exchange. This takes one address out of the pool; the next step takes the rest:

```
# 101_L2_LAB_attacker: a second shell
docker exec -it 101_L2_LAB_attacker bash
dhcp-starve 101-S1 1
```

The capture prints the whole exchange, one message per block. Find the server's **Offer**.

Question 2. From the captured **Offer**, give the name, number, and value of the option that identifies the server that sent it, and of the option that names the default gateway. State which of the two the attacker has to answer with a different value to become the victim's next hop.

Question 3. A client has to decide whether the **Offer** it just received came from the server its administrator runs. Name the field in your captured **Offer** that would let it decide, or state that the message carries none, and say what a client does when two different servers both answer its **Discover**.

Question 4. Your capture holds the whole exchange, not just the **Offer**. Name the message the tool sends after the **Offer** and the message the server answers it with, and state what the server does on receiving the first of those two that it had not already done when it sent the **Offer**.

3. **Starve the pool.** Now take the rest. In your second attacker shell:

```
# 101_L2_LAB_attacker: your second shell (the capture runs in the first)
dhcp-starve 101-S1          # runs until the server stops offering
```

Every fake client presents a hardware address the server has never seen and completes the whole handshake. The tool prints each address as it takes it. Read the total on its last line.

Question 5. The tool's last line gives the number of addresses it locked. Work out the size of the whole pool from the range in the addressing table, compare the two numbers, and account for the difference: name what was already holding the addresses the flood could not take.

Question 6. One host drained a pool meant for many. Explain what makes a hardware address the server has not seen before enough to be treated as a separate client, and name what the server would have had to check to tell twenty invented clients from twenty real ones.

4. **See the denial.** A machine the server has already met keeps its lease, so the damage only shows to one it has not. Give the victim's interface a new hardware address and ask with it:

```
# 101_L2_LAB_victim: back in the victim's shell
ip link set 101-S1 down
ip link set 101-S1 address 02:aa:bb:cc:dd:ee
ip link set 101-S1 up
udhcpc -i 101-S1 -q -n -t 5 -T 1
```

Record what `udhcpc` prints and how long it takes to finish.

Question 7. The victim had a working lease a minute ago, on the same cable and the same switch port, and now gets nothing. Record what `udhcpc` reported, and explain why the server still answers the hardware address it held before but not this one.

5. **Answer as the rogue.** The real pool is empty, so a server that still has addresses is the only one that can answer a new client.

Definition: Rogue DHCP server

A DHCP server run by someone other than the network's administrator, on a LAN that already has one. It is not a modified protocol or a special tool: it is a stock server, configured to answer, whose options point wherever its operator chooses.

You run a **rogue DHCP server** on the attacker in three steps. First, write its config. Every line but one is what an administrator would write for this subnet:

```
# 101_L2_LAB_attacker: your second shell
cat > /root/rogue.conf <<'EOF'
authoritative;                                # answer clients on this subnet
ping-check false;                             # offer at once, do not probe first
subnet 101.0.0.0 netmask 255.255.255.0 {
    range 101.0.0.150 101.0.0.200;             # no overlap with the real pool
    option subnet-mask 255.255.255.0;
    option routers 101.0.0.66;                 # the gateway it names to clients
}
EOF
```

That block is a here-document: every line between the two `EOF` markers is written into `/root/rogue.conf` instead of being run as a command. The range is `101.0.0.150–200` so the rogue never fights the real server for addresses; it only has to answer, and to set `option routers`.

Question 8. You now have two DHCP servers on one LAN with ranges that do not overlap, and one of them has no free addresses left. Predict the address and the default gateway the next new client will end up with, and name the line of the config above that decides each.

Second, create an empty lease database. `dhcpcd` refuses to start without a lease file to write to, and starting empty means the rogue's pool begins full:

```
# 101_L2_LAB_attacker: same shell
: > /root/rogue.leases
```

`:` is the shell's do-nothing command; `: > file` runs it with its output redirected to `file`, which truncates the file to zero bytes (creating it if absent) without printing anything.

Third, launch the rogue server on the LAN interface:

```
# 101_L2_LAB_attacker: same shell
dhcpcd -4 -q -cf /root/rogue.conf \
    -lf /root/rogue.leases -pf /root/rogue.pid 101-S1
```

`-4` is IPv4 only, `-q` quiets the startup banner, `-cf` names the config, `-lf` the lease database, `-pf` a pid file, and the trailing `101-S1` binds it to the lab interface.

Question 9. Suppose you had started this rogue without draining the real pool first. State which servers would answer the next **Discover**, what decides which **Offer** the client installs, and what draining the pool changed about that.

6. **See the hijack.** Ask for an address again, still as the client from step 4:

```
# 101_L2_LAB_victim: still the MAC you set in step 4
udhcpc -i 101-S1 -q -n -t 8 -T 1
ip -4 addr show 101-S1
ip route show default
```

Compare both against what you predicted.

Question 10. The victim's default gateway is now the attacker, and yet not everything it sends leaves through the attacker. State which destinations it now hands to the attacker and which it still delivers itself, and name the check the victim performs on each packet to decide between the two.

7. **Prove you are on-path.** Watch two destinations at once from the attacker: one on this subnet, one off it. Stop the DHCP capture in your first attacker shell with **Ctrl-C**, then:

```
# 101_L2_LAB_attacker: your first attacker shell
tcpdump -n -i 101-S1 'icmp and (dst host 101.0.1.1 or dst host 101.0.0.1)'
```

Send three packets to each from the victim. Nothing answers `101.0.1.1`; what matters is who receives the attempt:

```
# 101_L2_LAB_victim: in the victim's shell
ping -c3 101.0.1.1
ping -c3 101.0.0.1
```

Question 11. Record which of the two destinations your capture shows and which it does not. Explain the split from the decision the victim made for each destination, and name

what the attacker would have to do to the victim on top of the DHCP lease to see the traffic the capture missed.

Part 2: the defence

Both halves of the attack crossed the same device. A client cannot tell a real server from a rogue, and the real server cannot tell a real client from an invented one, but the switch in the middle knows something neither of them does: which physical port every message arrived on.

Definition: DHCP snooping

A switch feature that classifies each port as trusted or untrusted for DHCP. Server messages are forwarded only from trusted ports, and the same messages arriving on an untrusted port are dropped, so a host that starts answering DHCP is silenced by the switch rather than believed by the clients it lies to.

Definition: OpenFlow

The protocol and rule format Open vSwitch uses for its forwarding table. Each rule has a priority, a match on packet fields, and an action; the highest-priority matching rule wins. The action `NORMAL` hands the frame to the switch's ordinary learning logic.

DHCP snooping is the control built on that fact, and you build it here in two pieces with a repair in between. The first piece names the port the real server sits on and drops server messages arriving on any other, which silences the rogue. In between, you refill the pool the flood already emptied, because a rule added now does not give back an address that was leased ten minutes ago. The second piece binds each remaining port to the one hardware address behind it, which is what stops the flood: every client the flood invents is a hardware address that port has never had. The stock switch image ships no DHCP rules at all, so both pieces are **OpenFlow** rules you write on `br0`.

Open a shell on the switch and read the table it forwards with now:

```
# 101_L2_LAB_S1: the switch, opened from your own terminal
docker exec -it 101_L2_LAB_S1 bash
ovs-ofctl dump-flows br0
```

One rule, at priority 0, sending everything to `NORMAL`: the bridge forwards, and has no policy about DHCP of any kind. Now find the port numbers OpenFlow knows each host by:

```
# 101_L2_LAB_S1: in the same shell
ovs-vsctl get Interface 101-server    ofport
ovs-vsctl get Interface 101-victim    ofport
ovs-vsctl get Interface 101-attacker  ofport
```

Write the three numbers down; every rule below names one of them.

Question 12. Only one of the three hosts has any business sending an `Offer`. Give the port number you will name in the permit rule below, and say what a rule that permitted `Offers` from all three ports would have achieved against the attack you ran in Part 1.

Block the rogue: trust server replies from one port only

A DHCP client sends from UDP port 68 to port 67, and a server answers from 67 back to 68. A server's message is therefore recognisable at the switch by its direction alone, before anything inside it is read. Permit that direction from the trusted port and drop it everywhere else:

```
# 101_L2_LAB_S1: in the switch shell; substitute the server's ofport
ovs-vsctl set bridge br0 protocols=OpenFlow10,OpenFlow13
ovs-ofctl -O OpenFlow13 add-flow br0 "priority=300 in_port=<server-ofport> udp
    tp_src=67 tp_dst=68 actions=NORMAL"
ovs-ofctl -O OpenFlow13 add-flow br0 "priority=200 udp tp_src=67 tp_dst=68
    actions=drop"
```

Question 13. Both rules match on `tp_src=67` and `tp_dst=68` together, rather than on port 67 alone. Take a client's **Discover** and a server's **Offer** in turn, and say for each whether a rule matching any packet that involves port 67 would catch it. State what such a rule would do to the victim.

Question 14. The rogue server is still running on the attacker from Part 1. Predict which of the two rules its next **Offer** matches, and name the field of the packet that decides it. You will read the counter later and check.

Refill the pool the flood emptied

The rogue is silenced, but the real server still has nothing to hand out: every address it owns is leased to a client that does not exist. Try to take them back the obvious way first, by clearing the file the server keeps its leases in:

```
# 101_L2_LAB_server: the real server, opened from your own terminal
docker exec -it 101_L2_LAB_server bash
: > /var/lib/dhcp/dhcpd.leases      # empty the on-disk lease database
```

Then ask for an address as a machine the server has not met, and record what comes back:

```
# 101_L2_LAB_victim: in the victim's shell, with another new hardware address
ip link set 101-S1 down
ip link set 101-S1 address 02:aa:bb:cc:dd:ff
ip link set 101-S1 up
udhcpc -i 101-S1 -q -n -t 5 -T 1
```

Now restart the server, which starts it from the file you just emptied, and ask again:

```
# 101_L2_LAB_server: in the server shell
killall -x dhcpd
while pgrep -x dhcpd >/dev/null; do sleep 0.2; done
dhcpd -4 -q -cf /etc/dhcp/dhcpd.conf \
    -lf /var/lib/dhcp/dhcpd.leases -pf /var/run/dhcpd.pid 101-S1
pgrep -x dhcpd >/dev/null && echo "server up" || echo "dhcpd did NOT start"
```

The `while pgrep` line is not optional: `dhcpd` holds `101-S1` until it exits, so a second one started too early fails to bind and dies, leaving no server at all. `-q` silences that failure, which is why

the last line reads the result back instead of trusting it. If it reports failure, run the block again; the wait makes a second run safe.

```
# 101_L2_LAB_victim: ask again, the same hardware address as a moment ago
udhcpc -i 101-S1 -q -n -t 5 -T 1
ip -4 addr show 101-S1
ip route show default
```

Question 15. Record what the client got before the restart and after it, with the lease file empty on both attempts. Explain the difference, and say where the running server had been reading its list of allocated addresses from.

Stop the flood: bind each access port to one MAC

The pool is full again, and nothing yet stops the attacker emptying it a second time: the rules so far only inspect server messages. Look at what the flood puts on the wire. `dhcp-starve` invents a fresh source MAC for every fake client (a locally-administered `02:...` address), so a starved port is one port sourcing DHCP from dozens of different hardware addresses in a few seconds.

Definition: Port security

A rule that ties a switch port to the hardware address allowed to send from it and drops frames from any other source address on that port. It replaces the switch's default assumption, that a port's occupant is whoever it currently claims to be.

A real access port has one machine behind it, so **port security** states that machine's address once and refuses the rest. Read the two clients' current addresses:

```
# your own terminal: each line reads one client's MAC out of its own container
docker exec 101_L2_LAB_victim cat /sys/class/net/101-S1/address
docker exec 101_L2_LAB_attacker cat /sys/class/net/101-S1/address
```

Then, on the switch, permit each access port's bound MAC and drop client requests from any other source MAC on it. Here is the attacker's port:

```
# 101_L2_LAB_S1: in the switch shell; substitute the attacker's ofport and MAC
ovs-ofctl -O OpenFlow13 add-flow br0 "priority=270 in_port=<attacker-ofport> udp
    tp_src=68 tp_dst=67 dl_src=<attacker-mac> actions=NORMAL"
ovs-ofctl -O OpenFlow13 add-flow br0 "priority=260 in_port=<attacker-ofport> udp
    tp_src=68 tp_dst=67 actions=drop"
```

Add the same pair for the victim's port, with the victim's port number and the victim's MAC. Then make the forwarding policy explicit, rather than leaving the rest of the traffic to the bridge's own priority-0 rule:

```
# 101_L2_LAB_S1: everything that is not DHCP forwards as a normal switch
ovs-ofctl -O OpenFlow13 add-flow br0 "priority=10 actions=NORMAL"
```

Question 16. A request from a port's bound MAC matches both of that port's rules, the permit at priority 270 and the drop at 260. Predict what the flood and what the honest client

would each get if the two numbers were swapped, and say whether the rogue-blocking rules from the previous section would still hold in that case.

Question 17. Your two rules match DHCP client requests only (68→67); a frame of any other kind from an unbound source MAC on that port is not matched by either. Name something an attacker on the victim’s port can still do with a forged source MAC under these rules, and name the one thing it can no longer do.

Re-run the attack under the defence

Run each half again and read the counters. Take the flood first, on its own, so the number you read back reflects only what the real server gave up. Stop the rogue if it is still running from Part 1:

```
# 101_L2_LAB_attacker: your second attacker shell
pkill -f rogue.conf
dhcp-starve 101-S1
```

Read the tool’s last line. Then bring the rogue back and ask for a lease as the victim, with the hardware address the switch is bound to:

```
# 101_L2_LAB_attacker: same shell
: > /root/rogue.leases
dhcpcd -4 -q -cf /root/rogue.conf \
    -lf /root/rogue.leases -pf /root/rogue.pid 101-S1
```

```
# 101_L2_LAB_victim: the same hardware address the switch is bound to
ip addr flush dev 101-S1
udhcpc -i 101-S1 -q -n -t 5 -T 1
ip route show default
```

Unlike a rate limit, this result does not depend on timing. The flood cannot take one of the real server’s addresses at all, so there is no window a client has to avoid: it leases the same way during the attack as long after it. Read the counters on the switch:

```
# 101_L2_LAB_S1: back in the switch shell
ovs-ofctl -O OpenFlow13 dump-flows br0
```

Question 18. Give the `n_packets` counter on the priority-260 rule for the attacker’s port and on the priority-200 rule, and say which half of the attack each one counted. The priority-260 figure is far smaller than the number of addresses the flood took in Part 1: explain what the switch’s drop did to `dhcp-starve` that made it give up so early. Then say what this switch would have to keep doing against a flooder that did not give up, and name the action a production switch takes against a port that keeps violating its binding instead.

Checks: what “done” means

- **Attack works:** `dhcp-starve` drains the pool, a client with an unseen hardware address gets no lease from the real server, and once the rogue is running that client’s `ip route show`

`default` points at `101.0.0.66`. The attacker's `tcpdump` shows the victim's off-subnet traffic and not its on-subnet traffic.

- **Defence holds:** with the trusted-port rules and port security in place, the flood locks none of the real server's addresses, the victim leases from the real server with gateway `101.0.0.1` whether it asks during the flood or long after, and the `priority-200` (rogue) and `priority-260` (flood) counters both climb.

`scripts/status.sh` prints each host's addressing and the gateway the victim currently trusts, so you can read the before and after without shelling in. To undo your work, `scripts/reset.sh` stops the flood and the rogue, removes your flow rules, refills the real pool, and re-releases the victim, back to the Part 1 baseline.

Safety. Everything here lives on one isolated Open vSwitch bridge with no route off the host. The flood and the rogue server only ever reach the lab's own containers.

When you are done, tear the lab down. Choose **Teardown** in the TUI, or run `scripts/teardown.sh` directly, to remove the containers and their wiring so nothing is left running on your host. **Reset to baseline** only rewinds the attack and the defence; only **Teardown** deletes the lab.